Real-time Data Processing: Amazon Kinesis and Spark Streaming vs Batch

Assignment No. 4

1. Concepts

Batch processing collects data over a period, then processes the whole set in one run. Results are available when the run finishes, usually on a schedule (hourly or daily). It favours throughput and simple, repeatable jobs over the full data set.

Real-time (stream) processing handles each event or small micro-batch as it arrives. Results are available within seconds, and the job runs continuously. It favours latency, at the cost of more complex state handling, ordering and failure recovery.

Aspect	Batch	Real-time (stream)
Data	Bounded: a full file or table	Unbounded: events as they arrive
Result latency	Minutes to hours (schedule)	Milliseconds to seconds
Typical job	Full recompute, simple to reason about	Incremental, stateful, windowed
Failure handling	Re-run the job	Checkpoints and exactly-once sinks, more care needed
Cost pattern	Compute in bursts	Always-on compute

2. Advantages and popular tools

Approach	Key advantages	Popular tools
Batch	Simple, cheap per record, easy to rerun and audit, handles large history	Apache Spark (batch), Hadoop MapReduce, Hive, AWS Glue (batch ET
Real-time	Low latency, alerts and decisions while data is still fresh, continuous ingest	Amazon Kinesis Data Streams, Apache Kafka, Apache Flink, Spark Str

3. Practical use cases for real-time processing

- **Market stress alerts (this assignment).** Flag days when the VIX volatility index crosses a threshold, so a risk team can react the same day instead of at the next batch run.

- **Card fraud detection.** Score each transaction as it happens and block suspicious ones before authorisation completes. A nightly batch would detect fraud too late.

- **Industrial IoT monitoring.** Detect a sensor reading outside its normal range in seconds, so a machine can be stopped before damage occurs.

4. Pipeline design

Data: DataHub finance-vix daily file, 9,278 trading days from 2 January 1990 to 22 September 2026 (columns DATE, OPEN, HIGH, LOW, CLOSE). Both pipelines use the same enrichment and aggregation code (`common.py`), so differences in their output can only come from how the data arrives.

Real-time path (AWS design): producer → Amazon Kinesis Data Streams (one shard) → consumer → Spark Structured Streaming (two queries: per-event enrichment in append mode, calendar-month aggregates in update mode). The local stand-in sends the same events from the producer over a TCP socket to Spark.

Batch path: read the whole CSV once, enrich, and aggregate by calendar month.

5. What ran, and what did not

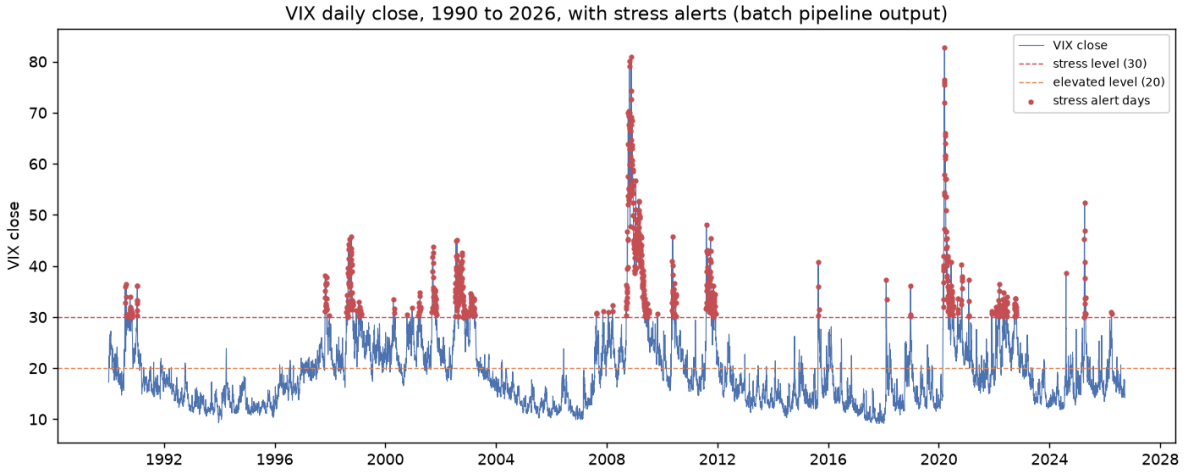
Component	Status on this machine
Batch pipeline (Spark, whole file)	Ran: 9,278 rows in 5.307 s (Spark 4.3.0, Java 22.0.2)
Shared logic and tests	Ran: 4 checks pass (regimes, monthly aggregates, loader, socket sink)
Real-time pipeline (Structured Streaming)	Not run here. Spark's streaming checkpoints need Hadoop native Windows binaries (winutils.exe), which
Amazon Kinesis ingestion	Not run. Needs an AWS account and credentials; a Kinesis shard incurs hourly charges. Code provided in
Real-time vs batch comparison	Ready (compare.py), waiting for a streaming run

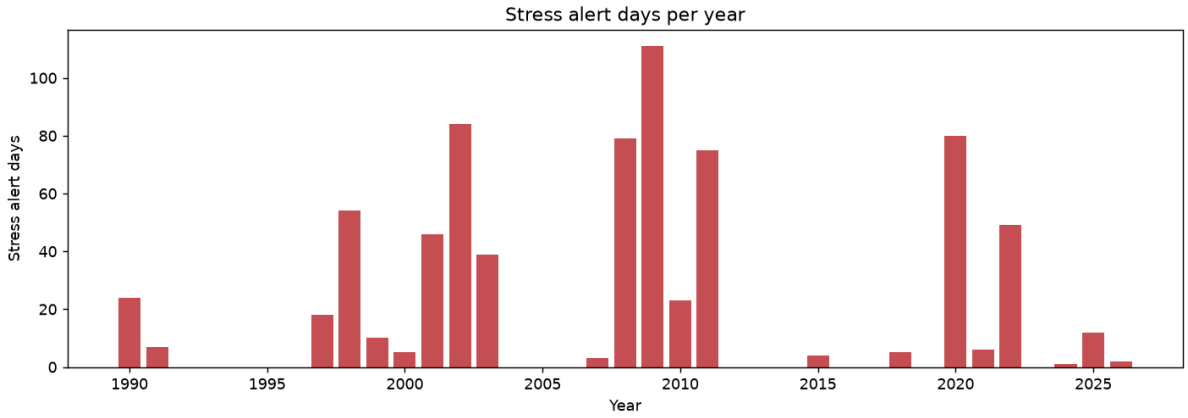
6. Results

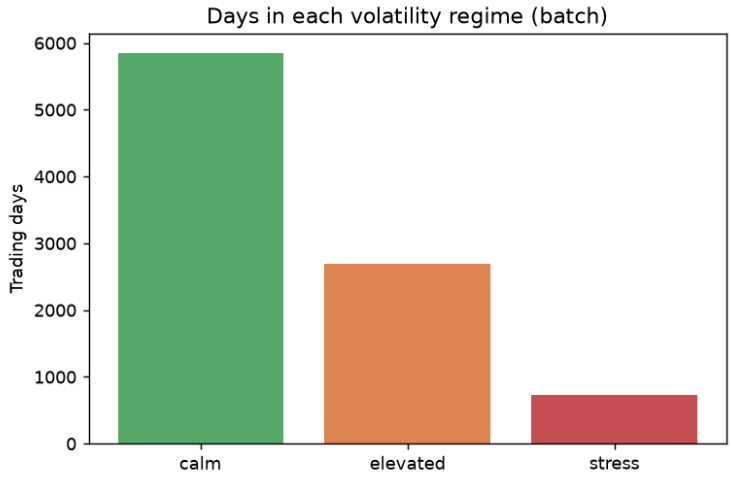
Batch results (produced by Spark on the whole file):

Regime	Trading days	Share
calm	5845	63.0%
elevated	2696	29.1%
stress	737	7.9%

Stress alert days (VIX close at or above 30): 737. The highest-alert years are 2009, 2002, 2020. The most recent month in the file is 2026-09.







Real-time vs batch comparison:

No streaming run is included, so the comparison is not reported yet. The table below lists what will be measured and where the batch side stands.

Measure	Batch	Real-time
Wall time for the whole file	5.307 s	not measured
Result latency per event	set by the schedule (not measured)	not measured
Per-event results identical to batch	reference	not measured
Monthly results identical to batch	reference	not measured

7. Limitations

- The VIX file is a daily series: it is not a true high-frequency feed, so latency is measured on replayed events.

- Spark's socket source, used for the local stand-in, is intended for testing and is not fault-tolerant.

- The Kinesis code was not run; its behaviour against AWS is untested in this project.

- The real-time comparison requires a streaming run, which is not included in this report yet.

8. References

- DataHub, Finance VIX daily data: https://datahub.io/core/finance-vix

- Cboe, VIX volatility index: https://www.cboe.com/tradable_products/vix/

- AWS, Amazon Kinesis Data Streams developer guide: https://docs.aws.amazon.com/streams/latest/dev/introduction.html

- Apache Spark, Structured Streaming programming guide: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html